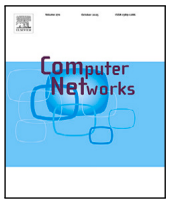
Computer Networks 286 (2026) 112399



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





Efficient and interpretable IoT botnet detection via feature selection and hyperparameter-optimized XGB☆

Elizabeth Antony *, Amit Kumar Roy

CSE-Cyber Security Department, Indian Institute of Information Technology Kottayam, Valavoor, Palai, 686635, Kerala, India

ARTICLE INFO

Keywords:
IoT security
Botnet detection
XGB
Feature selection
Explainable AI
Hyperparameter optimization

ABSTRACT

The Internet of Things(IoT) devices present a rapidly growing and high impact attack surface for botnet driven threats, yet resource constraints and diverse traffic patterns complicate reliable real time detection. This paper presents a practical, interpretable, and computationally efficient pipeline for IoT botnet detection that (i) converts packet captures to flow features, (ii) applies systematic Recursive Feature Elimination (RFE) to identify a minimal, high value feature set, (iii) trains and rigorously tunes an eXtreme Gradient Boosting(XGB) classifier using three complementary hyperparameter search strategies (Randomized, Step-by-Step Grid, and a heuristic Quick SmartSearch), and (iv) provides model interpretability via SHAP (SHapley Additive exPlanations) explanations. Experimental results demonstrate that the proposed framework consistently achieves superior detection performance in terms of accuracy, precision, recall, and F_1-score, while maintaining low inference latency. Experiments on a Mirai focused subset (2.34M packets) derived from the IoT Network Intrusion Dataset later balanced to 87,422 flows via Random UnderSampling(RUS) show that an eight feature XGB model retains near perfect discriminative power (Accuracy = 99.97%, F_1 = 99.97%) while achieving low per sample latency (inference ≈ 18.4 ms under the SmartSearch configuration). SmartSearch yields a compact hyperparameter configuration (n_estimators=75, max_depth=5, learning_rate=0.25, subsample=0.9, colsample_bytree=0.9) that matches top accuracy while substantially reducing training and inference costs relative to larger ensembles. SHAP analysis confirms that retained eight features align with established network security signals and also supporting model trustworthiness. Validation on independent IoT benchmarks (IoT-23, CTU-13, and BoT-IoT) further demonstrates transferability. Together, these results indicate a viable path to deployable, explainable IoT botnet detectors that balance accuracy, interpretability, and runtime efficiency.

1. Introduction

The swift expansion of IoT devices has fundamentally transformed the landscape of cybersecurity, introducing billions of new endpoints that serve as potential vectors for cyberattacks [1].

The IoT spans a diverse range of sectors beyond consumer applications such as smart homes, wearables, connected appliances, and personal digital assistants. Notable domains include healthcare, industrial systems, and smart cities, which represent prominent areas of deployment [2]. Furthermore, transportation and automotive IoT, agricultural systems, retail, and energy and utility networks constitute significant application areas. The scope is further extended to environmental monitoring, with each sector characterized by distinct functionalities and contexts of use [3].

Reflecting this broad adoption, the global IoT device count reached 18.8 billion in 2024, marking a 13% annual increase, with projections estimating further growth to 27 billion by 2025 and 40 billion by 2030. However, this unprecedented scale of connectivity introduces additional security risks, as the continual growth of the IoT ecosystem not only accelerates technological integration but also expands the attack surface available to cyber attackers. This expanded exposure has enabled the rise of increasingly large scale cyber threats, with IoT botnets emerging as one of the most critical and disruptive due to their ability to compromise vast numbers of devices and coordinate malicious activities [4].

A botnet is a collection of compromised devices, often referred to as ‘bots’, that are remotely controlled by a malicious entity known as the botmaster. IoT botnets exploit vulnerable devices to build extensive networks, enabling large scale Distributed Denial-of-Service (DDoS) attacks that can flood and incapacitate targeted systems or services. A DDoS attack is a cyberattack in which multiple compromised systems,

☆ This article is part of a Special issue entitled: ‘Cybersec Attacks and Defenses’ published in Computer Networks.

* Corresponding author.
E-mail addresses: elizabethantony.23phd21010@iiitkottayam.ac.in (E. Antony), amit@iiitkottayam.ac.in (A.K. Roy).

https://doi.org/10.1016/j.comnet.2026.112399
Received 24 October 2025; Received in revised form 14 April 2026; Accepted 16 May 2026
Available online 25 May 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

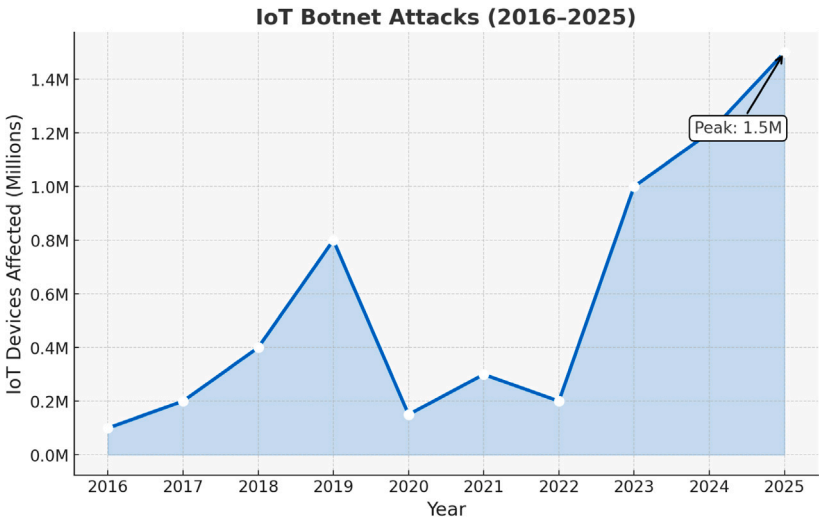


Fig. 1. Temporal Analysis of Botnet Attack Impact on IoT Infrastructure (2016–2025).

often part of a botnet, are coordinated to flood a target such as a server, network, or online service with excessive traffic [5]. The massive flood of malicious requests depletes the target's critical resources, such as bandwidth, processing power, or memory leading to significant performance degradation or even complete service disruption. DDoS attacks are particularly dangerous because they can disrupt critical services, impact businesses financially, and serve as a cover for additional malicious activities. As per NETSCOUT Systems, Inc.'s DDoS Threat Intelligence Report, in the first half of 2025 alone, there were over 8 million DDoS attacks globally, marking a dramatic intensification of activity in the digital realm. Fig. 1 demonstrates year by year analysis of botnet attack impact on IoT devices from the year 2016 to 2025.

IoT botnets may operate under two architectures:
(i) Centralized Command-and-Control (C&C) Architecture: In a centralized C&C architecture, all compromised IoT devices (bots) communicate with a central server controlled by the botmaster. The C&C server issues instructions (such as launching DDoS attacks, stealing credentials, spreading malware), and the bots execute them. This model allows attackers to easily manage and coordinate large botnets and creates a centralized dependency, where the failure of a single element can lead to a complete system breakdown. If the C&C server is identified and taken down by defenders, the entire botnet can be disrupted. Classic IoT botnets such as Mirai initially employed this centralized model due to its simplicity and efficiency in issuing commands [6].
(ii) Peer-to-Peer (P2P) Architecture: It refers to a decentralized model in which infected IoT devices act not only as bots but also as command relays. Unlike centralized architectures, this structure enables each compromised node to transmit commands to other bots, ensuring continuous operation even if one node fails. This decentralized design enhances resilience by eliminating a single point of failure, making it far more difficult for defenders to disable the botnet by targeting infrastructure [7]. P2P IoT botnets are harder to detect and dismantle because communication is distributed and often masked, allowing them to persist even if portions of the network are removed. Examples include Hajime and newer botnet variants that have evolved from Mirai [8].

Once compromised, IoT devices can be monetized via DDoS-as-a-service platforms, credential theft operations, or cryptocurrency mining, and are often utilized to propagate further infections. These attacks typically exploit device level vulnerabilities, such as weak authentication credentials (like default or easily guessed Telnet/Secure Shell (SSH) passwords) or remote code execution (RCE) flaws that enable unauthorized access and control.

Fig. 2 illustrates the full operational lifecycle of a botnet within the IoT ecosystem. It begins with a botmaster who controls the P2P and centralized C&C networks. These structures are adopted to conduct various malicious activities. The final targets are diverse and vulnerable IoT devices, represented by icons denoting typical smart home and connected technologies.

1.1. Research contribution

Existing IoT botnet detection approaches, including RFE-XGB based frameworks such as BotStop [9], have primarily focused on improving classification accuracy through feature selection, ensemble learning, or deep learning models. However, many of these methods either rely on high dimensional feature spaces, computationally expensive architectures, or lack systematic hyperparameter optimization, limiting their applicability in real time IoT environments.

In contrast, this work formulates IoT botnet detection as an *accuracy-efficiency trade-off problem*, where the objective is to maximize detection performance while minimizing feature dimensionality and computational cost. Unlike prior studies that treat feature selection, model training, and hyperparameter tuning as independent steps, the proposed framework integrates these components into a coherent optimization driven pipeline.

The main contributions of this work are summarized as follows:

- We propose a lightweight and optimized IoT botnet detection framework that jointly considers feature reduction and model efficiency, enabling deployment in resource constrained environments.
- We demonstrate that a minimal feature subset (8 features) can achieve near optimal detection performance (99.97%), establishing an empirical performance complexity trade-off that is not explicitly explored in prior work.
- We introduce a computationally efficient hyperparameter tuning strategy (SmartSearch) that achieves comparable accuracy to exhaustive search methods while significantly reducing training and inference time.
- We provide cross-dataset validation across IoT-23, CTU-13, and BoT-IoT datasets, demonstrating robustness and generalization across heterogeneous IoT traffic conditions.
- We incorporate SHAP-based explainability to validate that the selected features align with meaningful flow level network characteristics, improving interpretability and trust in model decisions.
- Compared to existing approaches, the proposed framework achieves a superior balance between accuracy, interpretability, and computational efficiency, making it suitable for real time IoT botnet detection.

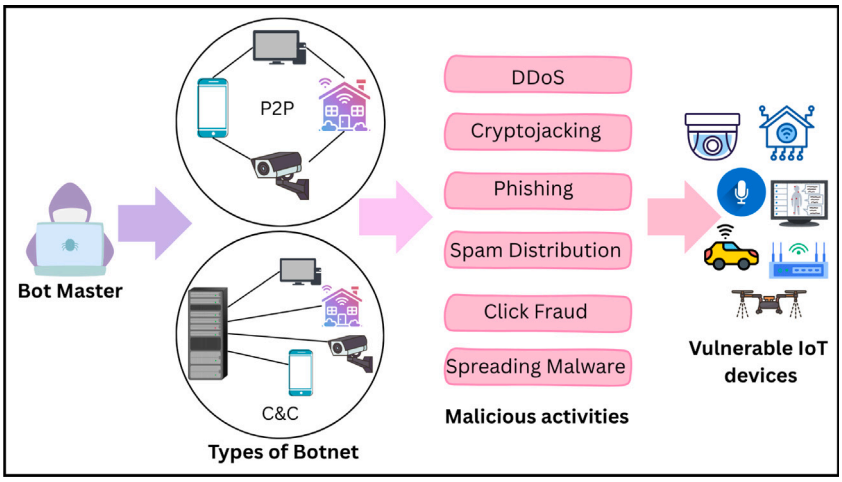


Fig. 2. IoT Botnet Structure and Malicious Activities.

1.2. Paper layout

Section 2 provides an overview of recent advancements in IoT botnet detection, focusing on machine learning techniques and explainable artificial intelligence methods. The proposed detection framework is introduced in Section 3, which covers data preprocessing, feature selection using RFE, and hyperparameter tuning of the XGB model. Details regarding dataset properties, preprocessing steps, and class imbalance handling are outlined in Section 4. Section 5 describes the experimental setup, evaluation metrics, and performance results, including cross-dataset validation and comparison with state-of-the-art methods. Model interpretability is explored in Section 6 through SHAP-based analysis, highlighting the significance of the selected features. Section 7 discusses the implications of the findings in terms of accuracy–efficiency trade-offs and deployment considerations, and includes a dedicated discussion on limitations and future research directions. Finally, Section 8 summarizes the study.

2. Related works

Botnet detection in IoT environments has been investigated through multiple paradigms such as signature based [10,11], anomaly based [12,13], DNS based [14,15], data mining [16,17], graph based [18,19], blockchain enabled [20,21], and deep learning–based [22,23] approaches. While these approaches provide diverse perspectives on addressing the botnet threat, the present work focuses specifically on classical machine learning(ML) methods, as they offer a practical trade-off by maintaining high detection accuracy, providing greater interpretability compared to deep learning models, and requiring lower computational resources, an important factor in resource constrained IoT environments.

Pokhrel et al. applied classical ML with feature engineering and the Synthetic Minority Oversampling Technique (SMOTE) on the BoT-IoT dataset, showing that the K-Nearest Neighbors (KNN) algorithm achieved stable detection performance with over 92% accuracy and high Receiver Operating Characteristic-Area Under the Curve (ROC-AUC) [24]. Building upon this, Lefoane et al. proposed a frequency based feature selection strategy, removing less informative attributes, and demonstrated superior performance (100% True Positive Rate (TPR), 0% False Positive Rate (FPR), 99.99% F1-score) on the IoT-23 dataset using Decision Tree (DT), Logistic Regression (LR), and Support Vector Machine (SVM) classifiers [25]. While both studies highlight the role of feature selection and class balancing in boosting performance, their reliance on a single dataset and limited evaluation metrics may constrain their generalizability.

Alani introduced BotStop, a packet level detection framework designed to mitigate latency issues present in flow based systems [9]. Using RFE, the feature set was reduced to seven, and classification with XGB achieved 99.76% accuracy and an F1-score of 0.9968. The model achieved a processing time of only 0.225 ms per packet, demonstrating exceptional real time capability. Additionally, SHAP values provided interpretability by ranking key features such as frame length, IP length, Time-To-Live (TTL), and port numbers. However, the absence of hyperparameter tuning leaves scope for further optimization.

Srinivasan & Deepalakshmi proposed an Ensemble Classifier Algorithm with Stacking Process (ECASP), which combines feature selection with stacked ensembles. Their approach achieved 94.08% accuracy on the Cyber Clean Center (CCC) dataset [26]. Velasco-Mata et al. addressed real time scalability by designing a lightweight DT based model using only four flow features extracted from short time windows [27]. Their framework achieved an F1-score of 0.926 with a processing latency of 0.007 ms per sample. These studies underscore the trade-off between feature simplicity and detection robustness, as relying on minimal, port based features can limit adaptability against advanced evasion strategies.

Gupta & Singh proposed a multi layer explainable ML framework structured into feature selection, detection, and classification modules [28]. RFE reduced the dataset to six optimal features, lowering computational costs while enhancing performance. Light Gradient Boosting Machine(LightGBM) achieved 99.98% accuracy in the detection layer, while Random Forest (RF) achieved 99.30% accuracy across seven botnet families in the classification layer. Random oversampling addressed class imbalance, and SHAP analysis identified influential features such as source address, traffic direction, and port numbers. Evaluations on the NCC-2 and CTU-13 datasets confirmed the model's robustness and interpretability, though the lack of hyperparameter optimization was noted as a limiting factor.

SB-Net is a recent ensemble based botnet detection framework employing a two-stage cascade Random Forest with ensemble feature selection on the CTU-13, NCC, and NCC-2 datasets. It achieves near perfect performance (99.99% accuracy, 99.79% macro-F1) [29]. However, its flow level representation and cascade design increase complexity compared to single stage classical ML approaches, potentially limiting efficiency in constrained IoT environments.

Taher et al. introduced a hybrid framework that combined Fisher Score Grasshopper Optimization Algorithm-KNN (FGOA-KNN) feature selection with an optimized neural network based on Harris Hawks Optimization (HHO) [30]. The approach achieved 98.7% accuracy on the N-BaIoT dataset, showcasing improved robustness and convergence. However, the use of a single dataset limits its broader applicability to other IIoT environments.

Lefoane et al. further advanced unsupervised pattern based feature selection, applying dominant value distribution and frequent itemset mining techniques to enhance efficiency. Their approach achieved a 100% TPR and 0% FPR while reducing computational overhead [31]. Nevertheless, the manual thresholding and limited attack diversity may reduce adaptability in dynamic traffic scenarios.

Lightweight ML classifiers have also gained traction due to their suitability for low resource IoT devices. Tikekar et al. applied Naïve Bayes (NB), KNN, and DT on the ISCX dataset, with NB achieving 90.62% accuracy [32]. This demonstrates the practicality of simple models, though their performance may degrade against more complex attack patterns. Expanding on this direction, Ali et al. introduced KSDRM, a stacked ensemble integrating KNN, SVM, DT, RF, and Multi Layer Perceptron (MLP) with LR as the meta learner [33]. On the UNSW-NB15 dataset, the model achieved near perfect accuracy (approx 99.9%), but again, its evaluation on a single dataset limits claims of transferability. Recognizing the importance of interpretability, Saied & Guirguis proposed an Explainable Artificial Intelligence (XAI) framework combining RF with Local Interpretable Model agnostic Explanations (LIME), SHAP, and rule extraction techniques [34]. Their model achieved high accuracy and enhanced transparency but was also evaluated on a single dataset, raising questions about scalability.

Ayo et al. developed Bot-FFX, a hybrid model combining Genetic algorithm (GA) and KNN to detect fast-flux botnets [35]. By leveraging Standard Deviation of Round Trip Time (SDRTT), Average Google Hits (AGH), and Global TTL Variation (GTV), the model achieved 99.18% accuracy with low false rates. However, the limited feature set and single dataset evaluation may reduce adaptability to sophisticated evasion tactics.

Gong & Liu employed LightGBM with Bayesian hyperparameter tuning and graph-based feature engineering on the CTU-13 dataset, achieving 96.62% accuracy with low FPR and FNR [36]. However, their reliance on CTU-13-specific features may restrict generalization. Ibrahim et al. proposed a multilayer behavioral framework combining semi-supervised filtering and supervised C&C detection [37]. Their model achieved an F1-score of 92% and a FNR of 1.5%, but evaluation on a single dataset limits its broader applicability.

Recent studies have focused on enhancing IoT botnet detection through advanced feature selection and ensemble learning techniques. For instance, Hossain and Islam [39] proposed a feature selection driven ensemble framework combining mutual information, correlation analysis, and PCA with multiple classifiers, achieving high detection accuracy along with improved interpretability using LIME. However, the reliance on ensemble models increases computational complexity, which may limit real time deployment in resource constrained IoT environments.

Similarly, Alshaeaa and Ghadhban [38] introduced a hybrid feature selection approach integrating correlation, generalized normal distribution optimization (GNDO), and lasso techniques, demonstrating improved detection performance across binary and multiclass tasks. Despite its effectiveness, the approach primarily focuses on feature optimization without explicitly addressing model efficiency or hyperparameter tuning.

These studies highlight the importance of feature optimization and accuracy improvement, but reveal a gap in jointly optimizing feature compactness, computational efficiency, and model simplicity for real time IoT deployment.

Unlike existing studies such as Alani [9], Gupta and Singh [28], and Ramadhan et al. [29], as well as recent works [38,39], the proposed framework explicitly addresses the joint optimization of feature compactness and model efficiency within a unified design. In contrast to ensemble or hybrid architectures, the proposed method achieves comparable performance using a single optimized model with a minimal feature set. Furthermore, the integration of systematic hyperparameter optimization and cross-dataset validation enhances generalization while reducing computational overhead, making it more suitable for real time IoT environments.

Table 1 summarizes recent IoT botnet detection studies, highlighting datasets, models, feature optimization techniques, performance, and limitations.

3. Proposed system overview

The proposed IoT botnet detection framework, illustrated in Fig. 3, is designed as a structured and multi stage pipeline that transforms raw network traffic data into an optimized, explainable machine learning model capable of accurately classifying malicious and benign IoT activity. The system operates through five stages: feature extraction, preprocessing, model training, feature selection and tuning, and explainability. Each stage progressively refines the data and model to achieve high accuracy, interpretability, and computational efficiency suitable for resource constrained IoT environments.

The process begins with the IoT Network Intrusion Dataset, consisting of multiple raw packet capture (pcap) files collected from simulated IoT network traffic. Raw packet captures are merged and converted into flow-level features, producing a structured dataset that captures statistical and protocol specific network behavior.

In the *dataset preprocessing stage*, the extracted features undergo a series of data refinement procedures. Host specific identifiers are removed to prevent information leakage. Class imbalance is addressed using a lightweight balancing strategy.

In the preliminary experimentation stage, the dataset is partitioned into training and testing subsets following an 80:20 ratio. Multiple classical ML classifiers are evaluated to identify a suitable base model. XGB is selected as the base classifier based on empirical performance. Accordingly, XGB is adopted as the reference model for further fine tuning.

In the *feature engineering and hyperparameter tuning stage*, the focus shifts to refining the XGB model for optimal performance. *RFE* is employed to identify and retain only the most discriminative features, reducing model complexity while preserving accuracy. Hyperparameters are optimized using multiple search strategies.

Finally, the *explainability and interpretation stage* integrates XAI techniques to provide transparency into model predictions. Model decisions are interpreted using SHAP based explanations, making the system suitable for practical deployment in IoT environments where trust is critical.

Overall, the proposed framework integrates efficient feature selection, robust model optimization, and explainable inference, resulting in a lightweight and deployment ready IoT botnet detection system suitable for real time operation on IoT gateways and edge devices. To provide a formal and theoretical foundation for the proposed framework, the system is mathematically formulated and analytically justified in the following subsections.

Algorithm 1 presents the overall workflow of the proposed IoT botnet detection framework, summarizing the key steps from data preprocessing and feature selection to model training, hyperparameter optimization, and explainability. This unified representation complements the detailed component level algorithms provided in Appendix and improves clarity and reproducibility.

3.1. Formal system model

In this study, IoT botnet detection is formulated as a supervised learning problem with an explicit objective of balancing detection performance and computational efficiency.

Let the dataset be defined as:

$$D = \{(x_i, y_i)\}_{i=1}^{N} \tag{1}$$

where $x_i \in \mathbb{R}^d$ represents the feature vector of the ith network flow, $y_i \in \{0, 1\}$ denotes the corresponding class label (benign or malicious), d is the total number of extracted features, and N is the number of samples.

Table 1
Summary of recent IoT botnet detection studies.

Study (Year)	Dataset	Technique/Model	Feature optimization	Acc. (%)/F_1	Key findings/contributions	Limitations
Pokhrel et al. 2021 [24]	BoT-IoT	KNN, classical ML + SMOTE	Manual feature engineering	92.0/–	Simple ML with data balancing improved stability.	Limited evaluation metrics and single dataset used.
Lefoane et al. 2021 [25]	IoT-23	DT, LR, SVM	Frequency based attribute pruning	99.99/0.9999	Frequency based feature removal improved accuracy and recall.	Manual thresholding.
Alani, 2022 [9]	IoT Network Intrusion Detection	XGB (packet level) + RFE + SHAP	Recursive Feature Elimination (7 features)	99.76/0.9968	Packet level RFE with SHAP interpretability; efficient for IoT devices (0.225 µs/packet).	No hyperparameter optimization performed.
Srinivasan & Deepalakshmi, 2023 [26]	Cyber Clean Center	Ensemble (ECASP)	Feature selection + stacking	94.08/–	Stacked ensemble achieved better generalization and robustness.	Evaluation limited to one dataset; static feature selection.
Velasco-Mata et al. 2023 [27]	Custom IoT dataset	Lightweight DT	4 traffic features	–/0.926	Real time detection with 0.007 ms/sample latency using minimal features.	Small feature set vulnerable to advanced evasion; port based dependency.
Taher et al. 2023 [30]	N-BaIoT	FGOA-kNN + IHHO-NN	Metaheuristic selection	98.7/–	Hybrid metaheuristic optimization improved robustness and detection accuracy.	Evaluation limited to a single dataset.
Ayo et al. 2023 [35]	Custom DNS dataset	GA-KNN (Bot-FFX)	Genetic feature selection	99.18/–	GA-KNN hybrid improved detection efficiency and reduced false rates.	Narrow feature set; limited dataset scope.
Gong & Liu, 2023 [36]	CTU-13	LightGBM + Bayesian tuning	Graph based features	96.62/–	Improved accuracy, low FPR/FNR.	Dataset specific features.
Ibrahim et al. 2024 [37]	CTU-13	Multilayer ML framework	Semi supervised filtering + supervised detection	92.0/–	Behavioral detection, low FNR.	Single dataset;
Gupta & Singh, 2024 [28]	NCC-2, CTU-13	Multi layer ML (LightGBM + RF) + SHAP	RFE (6 features) + oversampling	99.30/0.9926	Multi layer explainable ML with SHAP interpretability and class balance.	Lack of hyperparameter optimization.
Tikekar et al. 2024 [32]	ISCX	Naïve Bayes, KNN, DT	Basic feature filtering	90.62/–	Lightweight models suitable for low resource IoT environments.	Single dataset and limited feature diversity.
Alshaeaa & Ghadhban, 2024 [38]	UNSW-NB15	Classical ML (RF, AdaBoost, KNN, DT)	Hybrid FS (Correlation + GNDO + Lasso)	99.28/–	Hybrid feature selection improves detection performance across classification tasks.	Does not explicitly address model efficiency or hyperparameter optimization.
Ali et al. 2025 [33]	UNSW-NB15	Stacked ensemble (KNN, SVM, DT, RF, MLP + LR)	Integrated learners	97.94/0.9798	Ensemble learning captured complex behaviors with near perfect accuracy.	Single dataset evaluation.
Saied & Guirguis, 2025 [34]	IoT-23 subset	RF + SHAP + LIME	–	–/–	Explainable ML improved transparency and model trust.	Single dataset used.
Ramadhan et al. 2025 [29]	CTU-13, NCC, NCC-2	Cascade RF + ensemble feature selection	Rank aggregation (Borda)	99.99/0.9979	Two-stage cascade ensemble achieved state-of-the-art botnet detection across multiple datasets.	Flow-level features and cascade architecture increase complexity and deployment cost.
Hossain & Islam, 2025 [39]	IoT botnet datasets	Ensemble learning	MI + Correlation + PCA	99.99/–	Feature selection-driven ensemble with improved interpretability using LIME.	Increased computational complexity due to ensemble models.

3.1.1. Feature selection

Let $F = \{f_1, f_2, \ldots, f_d\}$ denote the full feature set. The goal of feature selection is to identify an optimal subset:

$$S \subseteq F, \quad |S| = k, \quad k \ll d \tag{2}$$

such that the selected subset S preserves discriminative information while reducing dimensionality.

The RFE process iteratively removes the least important features based on model derived importance scores:

$$S^{(t+1)} = S^{(t)} \setminus \{f_{least}\} \tag{3}$$

3.1.2. Model learning

Given the selected feature subset S, the classifier learns a mapping:

$$f_\theta : \mathbb{R}^k \to \{0, 1\} \tag{4}$$

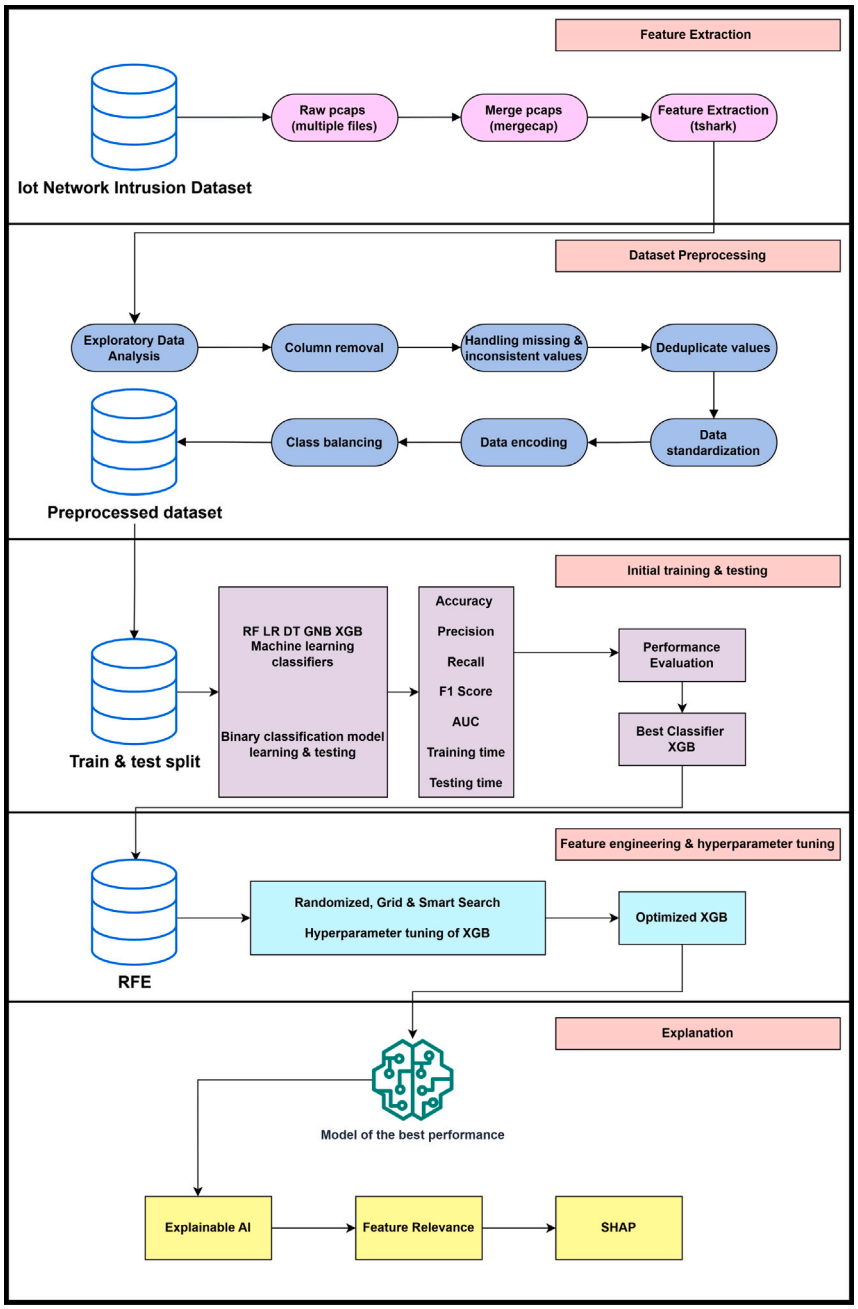


Fig. 3. Proposed Architecture of IoT botnet detection framework.

where θ represents the model parameters of the XGBoost classifier.

The prediction for a sample x_i is given by:

$$\hat{y}_i = f_\theta(x_i^S) \tag{5}$$

where x_i^S denotes the projection of x_i onto the selected feature subset S.

3.1.3. Hyperparameter optimization

Let Θ denote the hyperparameter space of the XGBoost model:

$$\Theta = \{n_{\text{estimators}}, \text{max_depth}, \eta, \text{subsample}, \text{colsample_bytree}\} \tag{6}$$

The objective is to find an optimal configuration $\theta^* \in \Theta$ that maximizes model performance while minimizing computational cost:

$$\theta^* = \arg\max_{\theta \in \Theta} \ \mathcal{L}(f_\theta(D)) \tag{7}$$

3.1.4. Objective function

The overall objective is formulated as a trade-off between detection performance and efficiency:

$$\max_{S,\theta} \quad \alpha \cdot \text{Performance}(S, \theta) - \beta \cdot |S| - \gamma \cdot C(\theta) \tag{8}$$

where:

- Performance(S, θ) represents classification performance (e.g., accuracy or F1-score),
- $|S|$ denotes the number of selected features,
- $C(\theta)$ represents computational cost (training/inference time),
- α, β, γ are weighting coefficients controlling the trade-off.

This formulation captures the key objective of the proposed framework: achieving high detection accuracy with minimal feature complexity and computational overhead, making it suitable for real time IoT environments.

Algorithm 1: Overall IoT Botnet Detection Framework

Input: Raw pcap files
Output: Optimized detection model f_{θ^*}
$Dataset \leftarrow$ Merge and extract flow-level features using Tshark;
Preprocess Dataset:;
  Remove host-specific identifiers and utility fields;
  Handle missing values (numeric $\rightarrow -1$, categorical $\rightarrow$ 'unknown');
  Convert protocol flags to decimal;
  Encode categorical features;
  Remove duplicate records;
Apply class balancing:;
  $Dataset \leftarrow$ Random UnderSampling($Dataset$);
Split dataset:;
  Divide into training and testing sets (80:20);
Feature selection using RFE:;
  Iteratively remove least important features;
  Obtain optimal feature subset S;
Model training:;
  Train XGBoost classifier using S;
Hyperparameter optimization:;
  Search over parameter space Θ;
  Obtain optimal parameters θ^*;
Final model:;
  Train optimized model f_{θ^*};
Evaluation:;
  Compute Accuracy, Precision, Recall, F1-score, AUC;
Explainability:;
  Apply SHAP for feature importance analysis;
return f_{θ^*};

3.2. Theoretical justification

This section provides analytical insights into the effectiveness and robustness of the proposed IoT botnet detection framework.

3.2.1. Effect of feature selection

Feature selection plays a critical role in improving model generalization by reducing redundant and irrelevant features. Let the original feature space be $\mathbb{R}^d$ and the reduced feature space be $\mathbb{R}^k$, where $k \ll d$. Reducing dimensionality decreases the risk of overfitting by limiting the hypothesis space of the model.

From statistical learning theory, the generalization error can be bounded as:

$$\mathcal{E}_{gen} \leq \mathcal{E}_{emp} + \mathcal{O}\left(\sqrt{\frac{h}{N}}\right) \tag{9}$$

where $\mathcal{E}_{emp}$ is the empirical error, h is the model complexity (related to feature dimensionality), and N is the number of samples. By reducing h through feature selection, the generalization error bound is improved.

3.2.2. Stability of RFE-based selection

RFE iteratively removes features with the lowest importance scores. Since importance is derived from the trained model, the selection process is guided by predictive relevance. This results in a stable subset of features that retains discriminative power while eliminating noise, as empirically observed in the performance plateau between 23 and 8 features.

3.2.3. Robustness of XGBoost

XGBoost constructs an ensemble of decision trees using gradient boosting, which minimizes a regularized objective function:

$$\mathcal{L} = \sum_{i=1}^{N} l(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t) \tag{10}$$

where $l(\cdot)$ is the loss function and $\Omega(f_t)$ is a regularization term controlling model complexity. This regularization improves robustness to noise and prevents overfitting, making XGBoost well suited for high dimensional network traffic data.

3.2.4. Effect of hyperparameter optimization

Hyperparameters such as tree depth, learning rate, and sampling ratios directly influence model bias and variance. The proposed SmartSearch strategy explores a constrained hyperparameter space to identify configurations that balance these factors. By avoiding excessively complex models (e.g., deep trees or large ensembles), the approach reduces variance while maintaining high accuracy.

3.2.5. Efficiency-accuracy trade-off

The proposed framework explicitly targets a trade-off between accuracy and computational efficiency. Reducing feature dimensionality and optimizing model parameters decreases both training and inference complexity. This leads to improved scalability and suitability for real time IoT environments without compromising predictive performance.

Overall, the combination of feature reduction, regularized boosting, and controlled hyperparameter optimization provides a theoretically grounded basis for the observed high accuracy, robustness, and efficiency of the proposed method.

3.3. Computational complexity analysis

This section presents a complexity analysis of the proposed IoT botnet detection framework in terms of time complexity, memory requirements, and communication overhead.

3.3.1. Time complexity

Let N denote the number of samples, d the number of original features, k the number of selected features ($k \ll d$), and T the number of trees in the XGBoost model.

Preprocessing: The preprocessing stage involves data cleaning, transformation, and feature extraction, resulting in a complexity of $\mathcal{O}(N \cdot d)$.

Feature Selection (RFE): RFE iteratively trains the model while eliminating features. Its complexity can be approximated as:

$$\mathcal{O}(d \cdot C_{model}) \tag{11}$$

where C_{model} represents the cost of training the base model.

Model Training (XGBoost): The training complexity of XGBoost is approximately:

$$\mathcal{O}(T \cdot N \cdot k) \tag{12}$$

which scales linearly with the number of samples and selected features.

Hyperparameter Optimization: Let H denote the number of hyperparameter configurations evaluated. The complexity is:

$$\mathcal{O}(H \cdot T \cdot N \cdot k) \tag{13}$$

The proposed SmartSearch strategy reduces H, thereby lowering the overall computational cost.

3.3.2. Memory requirements

The memory requirement is primarily dominated by storing the dataset:

$$\mathcal{O}(N \cdot k) \tag{14}$$

along with additional memory for storing the trained trees, which scales with the number of trees T. The reduction from d to k features significantly reduces memory usage.

3.3.3. Communication overhead

The proposed framework operates under a centralized training and edge level inference setting. Communication overhead mainly arises during data transfer and model deployment. Once deployed, inference is performed locally using a compact feature set, resulting in negligible runtime communication cost.

3.3.4. Discussion

Overall, the computational cost is dominated by the feature selection and hyperparameter optimization stages. However, the reduction in feature dimensionality and the use of a compact model significantly improve efficiency, making the framework suitable for real time IoT environments.

4. Dataset and preprocessing

The experiments in this study employ the IoT Network Intrusion Detection [40], which was developed for cybersecurity research in smart home environments. The dataset was constructed using traffic generated by two consumer IoT devices specifically, the SKT NUGU smart speaker (NU 100) and the EZVIZ C2C Mini O Plus 1080P Wi-Fi camera alongside laptops and smartphones operating within a shared wireless network. Network packets were captured via a wireless network interface card configured in monitor mode to ensure comprehensive traffic collection. To enhance data quality, wireless headers were removed using the Aircrack-ng toolkit. The final dataset contains 42 pcap files comprising a total of 2,985,994 packets, with 1,229,718 packets identified as malicious. The attack traffic includes traces generated by penetration testing tools (such as, Nmap) and traffic emulating the Mirai botnet, was initially generated on a laptop and subsequently modified to mimic communication from an IoT device. In a later version of the dataset, additional Mirai related traces were incorporated, including host discovery activities and Telnet brute force attack attempts.

Since this research focuses specifically on botnet detection, a subset of the dataset was selected that includes all available Mirai related files along with one benign file for baseline comparison. This selection resulted in 18 files (17 malicious + 1 benign). The subset contains a total of 2,339,621 packets, of which 1,037,977 are labeled as malicious. This selection preserves representative stages of Mirai botnet activity while retaining benign traffic to ensure realistic evaluation.

4.1. Dataset statistics

The **full dataset** consists of 42 pcap files containing 2,985,994 packets in total. Out of these, 1,229,718 packets are malicious, while 1,756,276 packets are benign, resulting in a malicious traffic ratio of 41.2%.

For the purposes of this study, a **selected subset** was constructed by including all Mirai related pcap files along with one representative benign pcap. This subset comprises 18 pcap files with 2,339,621 packets in total. Of these, 1,037,977 packets are malicious and 1,301,644 packets are benign, corresponding to a slightly higher malicious traffic ratio of 44.3%.

This choice of subset is motivated by two factors:

1. The Mirai botnet represents one of the most impactful and well-studied IoT botnet threats. Focusing on Mirai enables a detailed analysis of its scanning, Telnet brute force, and propagation phases.
2. The subset maintains a healthy distribution of malicious and benign traffic, which is beneficial for evaluating both detection capability and false positive rates in botnet detection models.

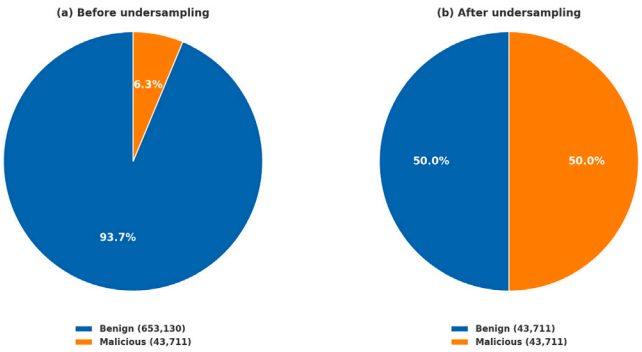


Fig. 4. Class distribution before and after RUS.

Thus, the selected subset is both **representative and balanced**, making it well suited for training and validating ML based IoT botnet detection systems.

Overall, the experiments in this study are conducted on a Mirai focused subset comprising 18 pcap files (approximately 2.34M packets), which are subsequently converted into flow level representations for machine learning analysis.

A comparison between the full dataset and the Mirai focused subset is presented in Table 2.

4.2. Data preprocessing and cleaning

The raw packet capture files were first merged using `mergecap` to produce a unified traffic trace. Flow level features were then extracted using `Tshark` and exported in structured `.csv` format for subsequent analysis. Although the selected attributes are derived from packet header fields, they are extracted and associated with each bidirectional flow record generated by Tshark; therefore, all classification experiments in this study are conducted at the flow level. Based on the official dataset description, each flow was labeled as either benign (0) or malicious (1), with manual verification performed to ensure labeling correctness.

To prepare the data for machine learning, several preprocessing steps were applied to improve data quality and prevent information leakage. Host-specific identifiers (e.g., MAC and IP addresses) and utility fields were removed to avoid device-dependent bias. Missing values were handled by replacing numeric fields with -1 and categorical fields with an *unknown* token. Protocol flag fields were converted from hexadecimal to decimal representation, and categorical attributes were label encoded. Duplicate records were removed to reduce redundancy and computational overhead.

RUS was selected to address class imbalance due to its simplicity, computational efficiency, and suitability for resource constrained IoT environments. Unlike SMOTE variants, RUS preserves the statistical characteristics of real network traffic and avoids introducing artificial flow patterns that may distort temporal or protocol level dependencies. Additionally, reducing the majority class lowers training and inference overhead, aligning with the deployment constraints of IoT gateways and edge devices.

After data cleaning, the dataset exhibited significant class imbalance. The applied sampling strategy produced a balanced dataset of 87,422 flows (43,711 benign and 43,711 malicious) across 23 features.

Fig. 4 illustrates the class distribution before and after balancing. The detailed preprocessing and balancing procedure is provided in Appendix - Algorithm 2.

Table 2
Comparison of Original IoT Intrusion Dataset and Selected Subset.

Dataset version	Files	Total packets	Malicious packets	Benign packets	% malicious
Full Dataset	42	2.99M	1.23M	1.76M	41.2%
Selected Subset (Mirai + 1 benign)	18	2.34M	1.04M	1.30M	44.3%

5. Experiments and results

5.1. Implementation setup

The experimental implementation was conducted using the Python programming language within the Visual Studio Code (VSCode) development environment. The experiments were executed on a system equipped with an Intel(R) Core(TM) i3-1005G1 CPU operating at 1.20 GHz, 8.00 GB of RAM (2667 MT/s), integrated Intel(R) UHD Graphics with 128 MB memory, and a total storage capacity of 1.14 TB.

5.2. Experimental design

The training and evaluation procedure was structured into multiple sequential phases:

1. **Baseline Training and Testing:** The dataset was split into training and testing sets using an 80:20 ratio. Feature selection and hyperparameter tuning were performed exclusively on the training data, and all reported results correspond to evaluation on the independent test set, thereby preventing information leakage between splits. A pipeline incorporating five distinct machine learning classifiers was developed, trained, and assessed to establish an initial performance benchmark.
2. **Feature Selection:** RFE was applied to identify an informative feature subset. The best performing model from the baseline phase was used as the base estimator for RFE.
3. **Evaluation after Feature Reduction:** A second training and testing phase was performed using the reduced feature set to examine the effect of feature selection on model predictive performance.
4. **Hyperparameter Optimization:** The selected classifier underwent hyperparameter optimization using a three-fold cross-validation strategy.
5. **External Dataset Validation:** Finally, model robustness and transferability were evaluated across multiple independent IoT botnet datasets. Validation was conducted on the IoT-23 [41] (specifically the CTU-IoT-Malware-Capture-34-1 subset), CTU-13 [42], and BoT-IoT [43] datasets to further assess out of sample performance and model applicability to unseen traffic conditions.

5.3. Evaluation metrics

In our experiments, we adopted standard binary classification metrics. Let TP, TN, FP and FN denote the numbers of true positives, true negatives, false positives, and false negatives, respectively. The metrics used include Accuracy, Precision, Recall and F1-score defined as follows.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{15}$$

$$Precision = \frac{TP}{TP + FP} \tag{16}$$

$$Recall = \frac{TP}{TP + FN} \tag{17}$$

$$F1\text{-}Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{18}$$

In addition to classification performance, computational efficiency was evaluated using training time and testing time, measured in seconds.

5.4. Results of baseline model training

During the baseline training and testing stage, a machine learning pipeline incorporating five classifiers: RF, LR, DT, Gaussian Naive Bayes(GNB), and XGB was evaluated.

The performance of the five baseline classifiers using the 23 selected features is summarized in Table 3. Among the evaluated models, XGB achieved the best overall performance, with the highest accuracy, perfect recall, and the highest AUC. XGB also attained a near perfect F_1-score. While RF and DT achieved comparable accuracy, their recall and AUC were marginally lower, making them less reliable for minimizing false negatives. LR and GNB offered faster training or testing times but exhibited lower accuracy and precision. XGB provided a favorable balance between detection performance and computational cost. **Therefore, XGB was selected as the final classifier for subsequent experiments in this study.**

To further validate classifier performance, ROC curves were generated using all 23 features. As shown in Figs. 5, 6 and 7 all classifiers demonstrated strong separability, with XGB (AUC = 1.0000) achieving a perfect ROC curve. RF and DT also performed strongly (AUC ≈ 0.9999), while GNB (AUC = 0.9823) and LR (AUC = 0.9658) achieved lower discriminative performance. The ROC results further confirm the robustness of XGB in distinguishing benign and malicious flows.

5.5. Feature selection using RFE

In this study, the XGB classifier initially trained on all 23 features achieved an F_1-score of 0.9997, which persisted as features were eliminated down to eight. The retained eight features were `frame.len`, `ip.proto`, `ip.ttl`, `ip.flags`, `tcp.dstport`, `tcp.window_size`, `udp.srcport`, and `udp.dstport`. A noticeable decline in F_1-score occurred when fewer than seven features were retained, supporting the choice of eight features as the optimal threshold. he detailed RFE procedure is provided in Appendix - Algorithm 3.

Fig. 8 illustrates this performance trend, highlighting the stability between 23 and 8 features, followed by a decline at 7 features and a sharp drop below 4 features. Comparative results for the 4-, 7-, and 8-feature configurations are discussed in the subsequent sections.

5.6. Post feature selection results

Among the evaluated classifiers, XGB achieved the best overall performance, maintaining the highest accuracy, perfect recall, and the highest AUC. While RF and DT achieved comparable accuracy and F_1-scores, XGB maintained the highest recall and AUC (1.0000). This is particularly important for botnet detection, where minimizing false negatives is critical.

Although XGB had already been selected as the base classifier in the initial evaluation phase, we included RF, DT, LR, and GNB in the post feature selection analysis for reference and completeness. The results show that XGB maintained its performance even with only eight features. LR delivered moderate results (accuracy 0.9557, precision 0.9589) with the advantage of a very low testing time (0.0048 s), while GNB was the fastest in training (0.0196 s) but exhibited the lowest accuracy (0.9021) and precision (0.8512).

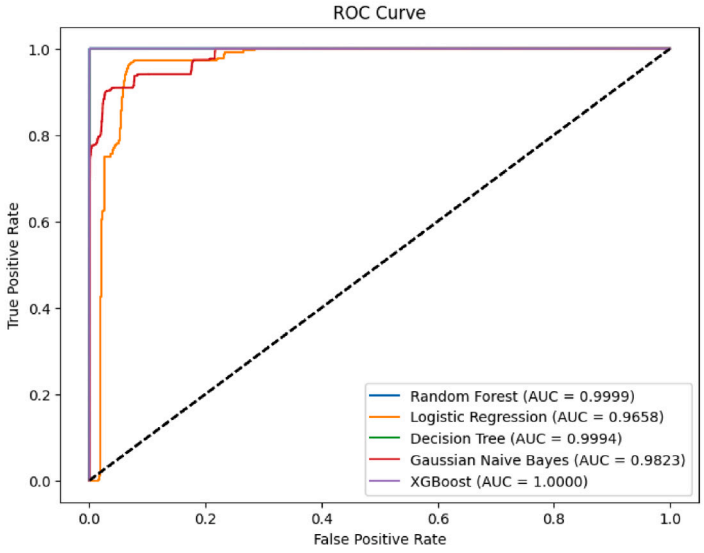
XGB provided a favorable balance between predictive performance and computational cost, with moderate training (0.2432 s) and testing (0.0218 s) times. These results further motivated the application of hyperparameter tuning to XGB.

Table 4 summarizes the classification results obtained after applying RFE to reduce the feature space to the most informative attributes.

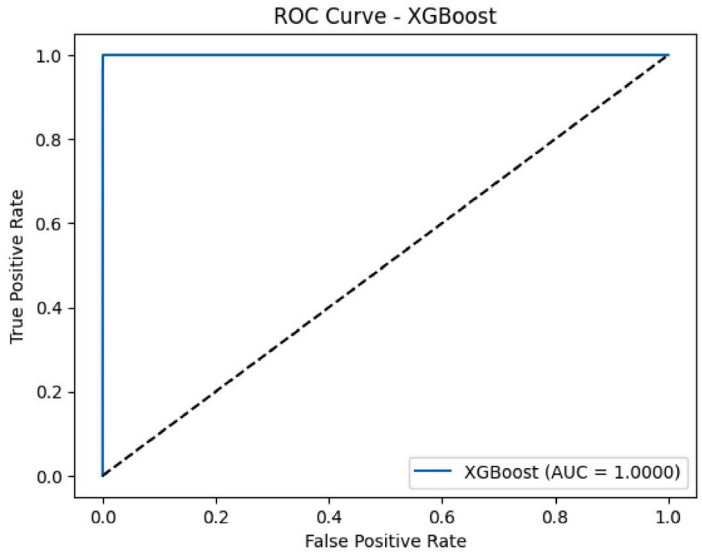
Table 3
Performance metrics of baseline classifiers using 23 selected features.

Classifier	A	P	R	F_1	AUC	T_rT (s)	TT (s)
RF	0.9995	**0.9994**	0.9997	0.9995	0.9999	6.5909	0.3316
LR	0.9477	0.9291	0.9693	0.9488	0.9658	7.7872	0.0295
DT	0.9994	0.9993	0.9994	0.9994	0.9994	0.2622	0.0181
GNB	0.8847	0.8263	0.9741	0.8942	0.9823	**0.0343**	**0.0119**
XGB	**0.9997**	0.9993	**1.0000**	**0.9997**	**1.0000**	0.3762	0.1345

Note: A = Accuracy, P = Precision, R = Recall, F_1 = F_1 core, T_rT = Training Time, TT = Testing Time. Bold values indicate the best performance per column.

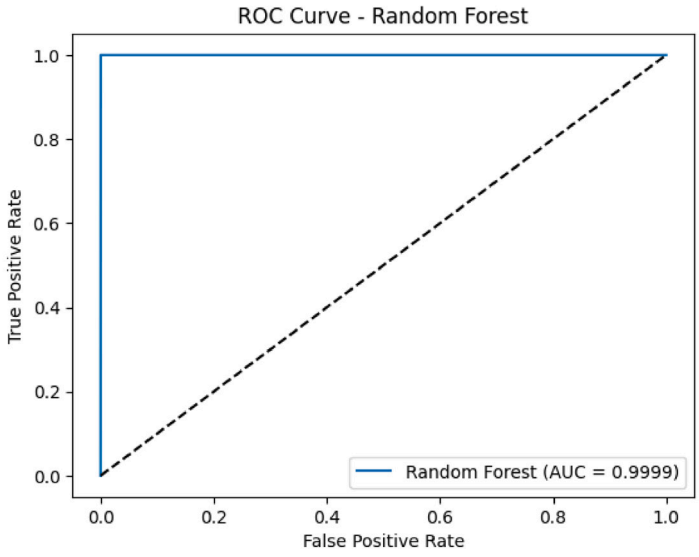


All Classifiers

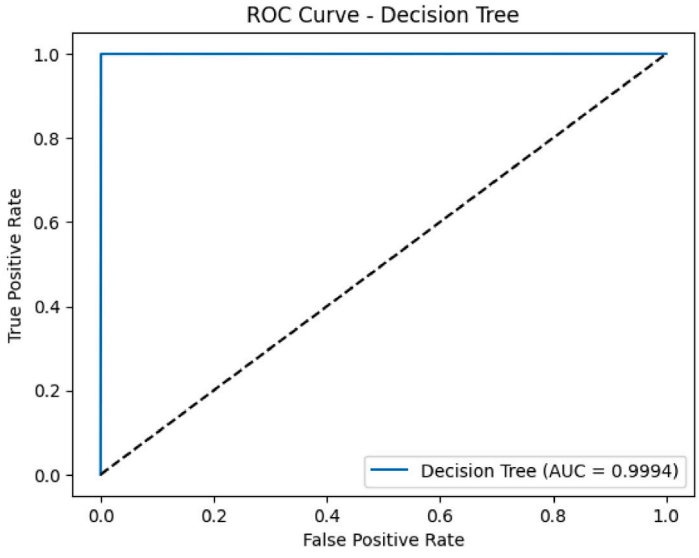


XGB

Fig. 5. ROC curve analysis of baseline classifiers using 23 features (part 1).



Random Forest



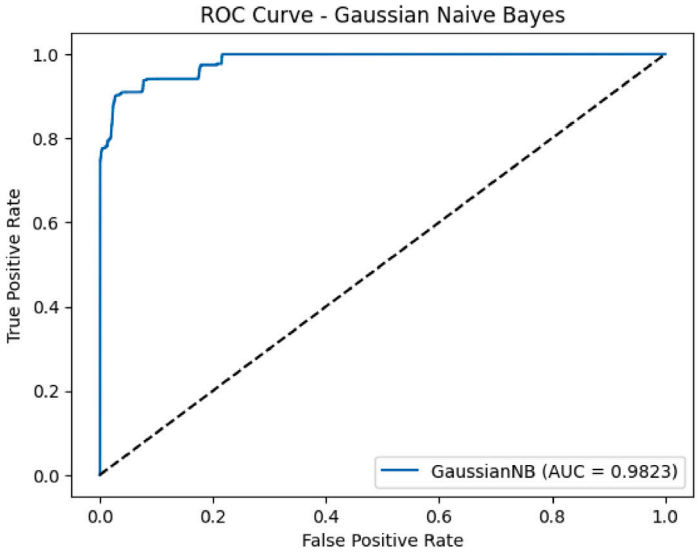
Decision Tree

Fig. 6. ROC curve analysis of baseline classifiers using 23 features (part 2).

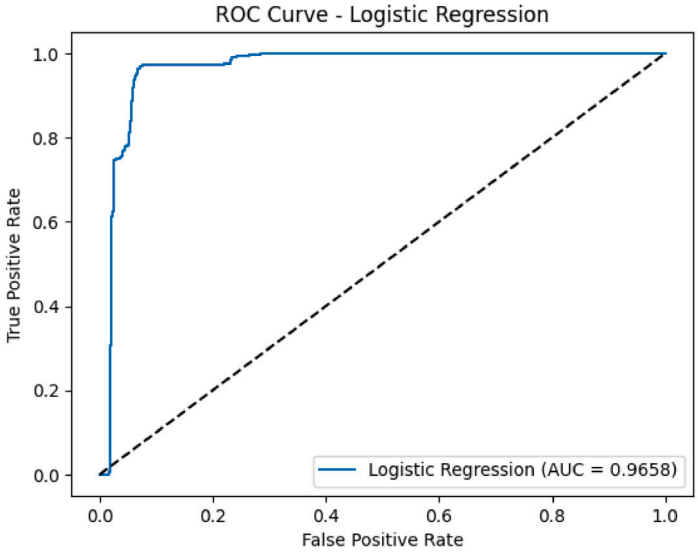
Table 4
Performance metrics of baseline classifiers after feature selection using 8 features.

Classifier	A	P	R	F_1	AUC	T_rT (s)	TT (s)
RF	0.9997	**0.9995**	0.9999	0.9997	0.9999	2.6852	0.2311
LR	0.9557	0.9589	0.9523	0.9556	0.9919	0.7016	**0.0048**
DT	0.9997	0.9995	0.9999	0.9997	0.9998	0.0696	0.0041
GNB	0.9021	0.8512	0.9746	0.9087	0.9803	**0.0196**	0.0103
XGB	**0.9997**	0.9994	**1.0000**	**0.9997**	**1.0000**	0.2432	0.0218

Note: A = Accuracy, P = Precision, R = Recall, F_1 = F_1-score, AUC = Area Under Curve, T_rT = Training Time, TT = Testing Time. Bold values highlight the best results per metric.



Gaussian Naïve Bayes



Logistic Regression

Fig. 7. ROC curve analysis of baseline classifiers using 23 features (part 3). XGBoost achieves perfect separability (AUC = 1.0000), confirming its superior classification performance.

5.7. Hyperparameter tuning of XGB

Three hyperparameter tuning strategies were applied to the XGB classifier trained on the eight features selected by RFE: Randomized Search, Step-by-Step Grid Search, and SmartSearch.

Randomized Search explored a broad hyperparameter space covering tree depth, learning rate, ensemble size, and sampling ratios, achieving a classification accuracy of 0.9997.

Step-by-Step Grid Search focused on a narrower hyperparameter range and achieved the same classification accuracy (0.9997) as Randomized Search.

SmartSearch employed heuristic, domain informed hyperparameter combinations and achieved a classification accuracy of 0.9997. The detailed tuning procedure is provided in Appendix - Algorithm 4.

Table 5
Performance comparison on the eight feature dataset.

Method	T_rT (s)	TT (s)	T_{tot} (s)	A	Inf (ms)
Randomized	26.31	0.0314	26.34	0.9997	31.4
StepByStep	25.80	0.0328	25.83	0.9997	32.8
SmartSearch	**14.09**	**0.0184**	**14.11**	0.9997	**18.4**

Note: T_rT = training time, TT = testing time, T_{tot} = total time, A = accuracy, Inf = inference time (ms).

The results in Table 5 summarize the runtime and accuracy outcomes of the three hyperparameter search strategies evaluated on the 8-feature dataset. All three strategies achieved the same classification accuracy (0.9997).

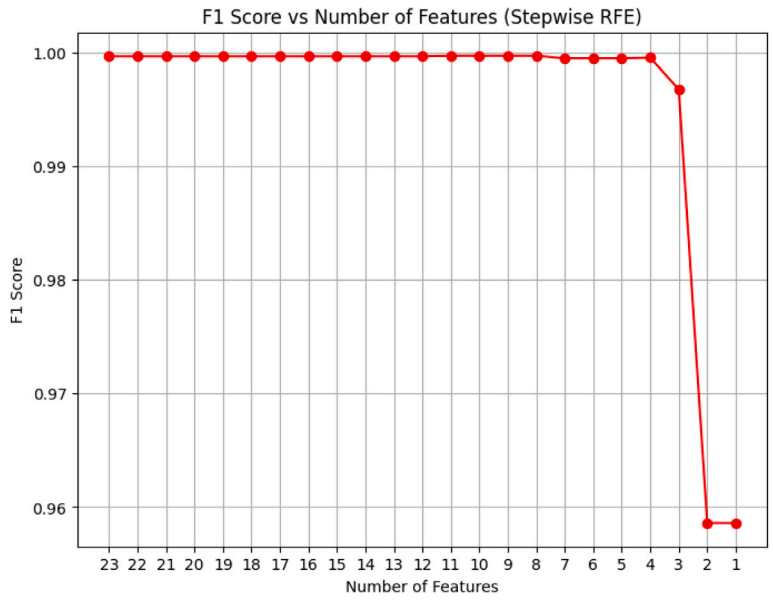


Fig. 8. F_1-score vs. Number of Features obtained using stepwise RFE. The plot highlights the stability of the F_1-score between 23 and 8 features, followed by a gradual decline at 7 features and a sharp drop below 4 features.

Table 6
Best hyperparameter combinations found by each search strategy (8 selected features).

Method	n_{est}	d_{max}	η	Sub	ColSub
Randomized	200	6	0.30	1.0	0.9
StepByStep	200	6	0.30	1.0	1.0
SmartSearch	**75**	**5**	**0.25**	**0.9**	**0.9**

Note: n_{est} = n_estimators, d_{max} = max_depth, η = learning rate, Sub = subsample, ColSub = colsample_bytree.

Table 6 lists the best hyperparameter configurations identified by each method. Randomized and Step-by-Step searches converged to larger ensemble configurations with deeper trees. SmartSearch identified a more compact configuration with fewer estimators and shallower trees. To further examine the interaction between feature dimensionality and hyperparameter optimization behavior, the three tuning strategies were also evaluated across reduced feature subsets (4, 7, and 8 features). The comparative results are summarized in Table 7.

Table 7 shows that all search strategies maintain consistently high accuracy across feature subsets, while computational cost varies with both model complexity and feature dimensionality. Notably, SmartSearch retains the lowest training and inference time in the 8-feature configuration, confirming its efficiency advantage under the selected optimal feature set. These results further support the selection of the 8-feature representation combined with SmartSearch tuning for deployment oriented scenarios.

5.8. Cross-dataset validation

To evaluate the generalization capability of the proposed framework, the optimized XGB configuration was first tested on the IoT-23 dataset.

Table 8 summarizes the classification performance achieved on this external dataset. The results show that the trained model maintains high detection performance when evaluated on previously unseen traffic, achieving near perfect precision, recall, and F_1-score on the IoT-23 dataset.

To further assess cross-dataset generalization, the proposed detection pipeline was additionally evaluated on two widely used IoT botnet benchmarks: CTU-13 and BoT-IoT. For each dataset, the same preprocessing pipeline, RFE based feature selection procedure, and optimized XGB hyperparameters were applied, ensuring a consistent evaluation protocol while allowing dataset specific feature subsets to be selected.

The comparative results are summarized in Table 9. On the CTU-13 dataset, the model achieved an F_1-score of 0.9467 on a balanced test set, indicating robust detection capability despite differences in traffic characteristics and botnet families compared to the Mirai-focused training data.

On the BoT-IoT dataset, the framework achieved near perfect performance with an accuracy of 1.000 and F_1-score of 0.9975, demonstrating strong transferability across heterogeneous IoT attack scenarios and network environments.

These cross-dataset results indicate that the proposed RFE-XGB detection framework maintains strong performance across multiple IoT botnet datasets with differing characteristics.

5.9. Comparison with existing work

To contextualize the effectiveness of the proposed approach, a quantitative comparison with existing botnet detection methods reported in the literature was conducted.

Table 10 summarizes the performance of representative benchmark studies evaluated on similar datasets. The proposed framework achieves higher or comparable detection accuracy and F_1-score relative to existing methods while relying on a reduced feature set and a compact model configuration. This comparison suggests the effectiveness of combining targeted feature selection with optimized gradient boosting for IoT botnet detection.

5.10. Evaluation across network scenarios and parameter configurations

To further validate the robustness of the proposed framework, additional analysis was conducted across multiple datasets and parameter configurations.

Experiments were performed on multiple benchmark datasets, including IoT-23, CTU-13, and BoT-IoT, representing diverse network environments and traffic patterns. These datasets capture variations in attack behavior, traffic distribution, and feature characteristics, enabling evaluation under different network scenarios.

Table 7
Comprehensive comparison of hyperparameter search strategies using 4, 7, and 8 selected features.

Method	T_rT (s)			T_{tot} (s)			Inf (ms)			A		
	4f	7f	8f	4f	7f	8*f*	4f	7f	8f	4f	7f	8f
Randomized	21.85	24.01	26.31	21.87	24.03	26.34	18.1	27.6	31.4	0.9995	0.9995	0.9997
StepByStep	8.39	9.36	25.80	8.41	9.37	25.83	13.4	9.5	32.8	0.9995	0.9995	0.9997
SmartSearch	11.84	12.83	14.09	11.87	12.86	14.11	30.8	30.4	18.4	0.9995	0.9995	0.9997

Note: T_rT = training time, T_{tot} = total time, Inf = inference time, A = accuracy.

Table 8
Results from testing the XGB classifier on the IoT-23 dataset.

Metric/Measurement	Value
Accuracy	0.999568
Precision	1.000000
Recall	0.999529
F1-score	0.999764
Inference time (total)	0.0042 s
Inference time per sample	0.0009 ms

Table 9
Cross-dataset evaluation of the proposed RFE-XGB detection framework.

Dataset	Accuracy	F_1-Score
Proposed dataset (IoT Network Intrusion)	0.9997	0.9997
IoT-23	0.9996	0.9998
CTU-13	0.9467	0.9467
BoT-IoT	1.000	0.9975

In addition, the impact of parameter configurations was analyzed through systematic hyperparameter tuning of the XGBoost model. Key parameters such as the number of estimators, learning rate, and tree depth were varied to evaluate their influence on detection performance. The results demonstrate that the proposed framework maintains stable and high performance across different parameter settings, indicating strong robustness and adaptability.

Furthermore, the effect of feature dimensionality was evaluated by analyzing model performance across varying numbers of selected features. The results show that a compact feature subset can achieve near optimal performance, highlighting the efficiency of the proposed feature selection strategy.

5.11. Statistical performance analysis

To ensure the reliability and consistency of the proposed framework, statistical evaluation metrics were incorporated into the experimental analysis. The model was evaluated over 10 independent runs using the optimized hyperparameters identified through the SmartSearch strategy.

The proposed model achieves an average accuracy of 99.97% ± 0.014, computed over 10 independent runs, demonstrating high stability and robustness.

The low standard deviation indicates that the model is stable and not sensitive to variations in training data or initialization. Table 11 further illustrates the consistency of the model performance across multiple runs, confirming its statistical reliability.

5.12. Ablation study

To evaluate the contribution of each component in the proposed framework, an ablation study was conducted by systematically removing key components and analyzing their impact on performance and computational efficiency.

The results show that all configurations achieve comparable detection accuracy. However, the proposed full model achieves this performance using only 8 selected features, compared to 23 features when feature selection is not applied.

In terms of computational efficiency, removing feature selection increases the execution time from 0.59 s to 0.78 s, demonstrating that RFE significantly reduces computational overhead. Although the default XGBoost model exhibits lower runtime (0.44 s), it lacks optimized parameter configuration and may not generalize as effectively across different datasets.

Overall, the proposed framework achieves an effective balance between accuracy, feature compactness, and computational efficiency, making it suitable for real time IoT botnet detection.

Table 12 presents the results of the ablation study, highlighting the contribution of each component of the proposed framework.

6. Model explainability

To maximize the credibility of the proposed botnet detection system, we emphasize the explainability of the trained XGB model. Model transparency is increasingly recognized as a critical component of the machine learning process based security research, as it helps assess whether high predictive accuracy is associated with meaningful, interpretable relationships rather than opaque black-box behavior.

In this work, we employ **SHAP**, a widely adopted framework for interpreting model decisions. Grounded in cooperative game theory, SHAP explains model predictions by quantifying the marginal impact of each feature across every possible subset of features. This approach provides a theoretically grounded and consistent framework for model agnostic explanations. Given that our model is tree based, we employ the *TreeExplainer*, which enables an efficient and exact computation of SHAP values with polynomial time complexity.

Fig. 9 depicts the SHAP summary plot corresponding to the eight selected network traffic features, ordered according to their overall importance in the prediction process. The SHAP value is plotted along the horizontal axis, reflecting the magnitude and direction of each feature's contribution to the model output. Values on the left (negative) are associated with benign predictions, whereas values on the right (positive) indicate a shift toward the attack class. The color scheme reflects feature intensity, with red corresponding to higher feature values and blue to lower values.

Among the analyzed features, `frame.len` (packet length) emerges as the most influential. Larger packet sizes are consistently associated with strongly positive SHAP values, leading the classifier to predict malicious behavior. This observation is consistent with the intuition that adversaries often exploit larger packet sizes in data exfiltration or C&C activities to maximize throughput. Conversely, smaller packets, which are characteristic of routine benign traffic such as DNS or ICMP queries, contribute negatively to the prediction, aligning with the benign class.

The second most influential feature is `ip.flags`. Interestingly, higher values of this field are associated with negative SHAP contributions, steering predictions toward benign traffic, while lower or manipulated values increase the likelihood of attack classification. This trend reflects well established networking practices: legitimate traffic typically adheres to standard flag settings, such as the "Do Not Fragment" (DF) flag, while malicious traffic frequently employs unconventional or unset flags to evade inspection mechanisms.

`tcp.dstport` also plays a crucial role. The SHAP distribution shows that connections involving standard service ports (such as 80 for HTTP or 443 for HTTPS) tend to be classified as benign, while traffic directed toward high or non standard ports receives positive SHAP

Table 10
Performance comparison of the proposed system with published works.

Work	Algorithms	Dataset	A (%)	F_1 (%)
Our Proposed Work	XGB	IoT Network Intrusion Detection	**99.97**	**99.97**
Ali et al, 2025 [33]	KSDRM Stacking	UNSW-NB15	97.94	97.98
Ramadhan et al, 2025 [29]	Cascade RF + Ensemble FS	CTU-13, NCC, NCC-2	99.99	99.79
Gupta & Singh, 2024 [28]	LGBM, RF	NCC-2, CTU-13	99.30	99.26
Alani, 2022 [9]	XGB	IoT Network Intrusion Detection	99.76	99.68
Gong & Liu, 2022 [36]	LGBM, Bayesian Opt.	CTU-13	96.62	86.71
Pokhrel et al. 2021 [24]	KNN	BoT-IoT	92.10	–
Ibrahim et al. 2021 [37]	K-means, KNN	CTU-13	92.20	91.51

Note: A = Accuracy, F_1 = F_1-score.

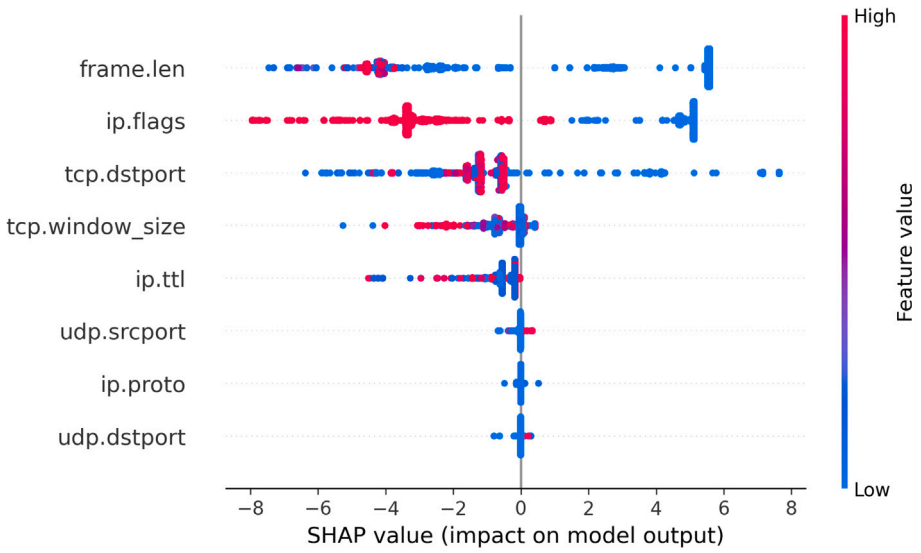


Fig. 9. SHAP summary plot of the eight selected features, showing feature importance and impact on classification outcomes.

Table 11
Statistical evaluation of the proposed model over 10 independent runs.

Run	Accuracy
1	0.9998
2	0.9997
3	0.9998
4	0.9999
5	0.9997
6	0.9996
7	0.9996
8	0.9998
9	0.9994
10	0.9998
Mean Accuracy	**0.9997**
Standard Deviation	**0.000136**

Table 12
Ablation study of the proposed framework.

Configuration	Features	Accuracy (%)	Time (s)
Full model (RFE + SmartSearch + XGB)	8	99.97	0.59
Without feature selection	23	99.96	0.78
Without hyperparameter tuning	8	99.97	0.44

values, indicating a higher probability of attack. This aligns with the well documented strategy of adversaries leveraging obscure ports for covert communication or exploiting services not commonly monitored.

The `tcp.window_size` feature displays a more nuanced impact. Smaller window sizes tend to correspond to benign traffic, whereas larger or irregular values exhibit mixed behavior. This observation may relate to the deliberate manipulation of TCP window parameters in certain flooding or DDoS attacks, where abnormal flow control settings are used to disrupt communication.

Similarly, the `ip.ttl` feature shows a discernible pattern: packets with unusually high TTL values are more likely to be flagged as malicious, while lower values often correspond to benign activity. This finding reflects known attack strategies where TTL values are artificially manipulated to prolong packet lifetimes or circumvent detection heuristics based on hop counts.

In contrast, features such as `udp.srcport` and `udp.dstport` exhibit relatively lower global importance, as reflected by the concentration of their SHAP values around zero. Nonetheless, when anomalies occur such as traffic directed to non standard UDP ports they still contribute positively to attack classification. This behavior corresponds to UDP based attack vectors such as amplification or covert tunneling, where adversaries deliberately select unusual port configurations.

Finally, `ip.proto`, though ranked lowest in global importance, provides subtle yet useful distinctions in classification. This feature encodes the type of protocol encapsulated within the IP packet (TCP, UDP, ICMP, IGMP). While in benign network traffic, TCP and UDP

dominate due to their prevalence in web browsing, email, and streaming applications, deviations from this norm can provide indicators of suspicious activity. For instance, a disproportionately high frequency of ICMP packets may be associated with scanning or ping flood attacks, whereas abnormal use of IGMP has been linked to reconnaissance and traffic amplification strategies. Additionally, malicious actors may craft packets with uncommon or undefined protocol numbers to evade traditional firewall or IDS filtering mechanisms. Although these occurrences are relatively rare compared to variations in packet length or TCP flag usage, their presence contributes an additional layer of discrimination between benign and malicious flows.

In summary, the SHAP analysis indicates that the classifier primarily relies on traffic attributes that are consistent with established network security principles. This supports the interpretability of the learned decision patterns and suggests that the model captures relevant discriminative structure within the data. By making model decisions transparent, the proposed solution becomes not only accurate but also interpretable, a property important for practical deployment in real-world botnet detection environments.

7. Discussion

The experimental results show that high botnet detection performance can be achieved using a compact and carefully optimized feature set. Although classification accuracy remained high across different feature subsets, the 8-feature configuration provided the most balanced trade-off between detection accuracy and computational efficiency. This indicates that protocol and transport layer attributes capture sufficient discriminative information for effective IoT botnet detection while enabling reduced model complexity.

Hyperparameter tuning further demonstrated that comparable predictive performance can be obtained using different optimization strategies, but with substantially different computational costs. While Randomized and Step-by-Step Grid Search converged to larger and more computationally expensive models, SmartSearch identified a more compact XGB configuration that preserved accuracy while reducing training time and inference latency. This highlights the importance of efficiency aware optimization for real time and resource constrained IoT environments.

In addition to inference latency, model size and memory footprint are important considerations for deployment on IoT gateways and edge devices. The proposed framework employs a compact XGB configuration (75 trees with a maximum depth of 5) and only eight input features, which inherently limits both model storage requirements and runtime memory usage compared to larger ensembles. While detailed hardware specific profiling is outside the scope of this study, the observed low inference latency and compact model structure indicate suitability for resource constrained environments.

Evaluation on the IoT-23 dataset confirms the ability of the proposed framework to generalize to previously unseen traffic. In comparison with existing approaches, the proposed method achieves competitive or superior detection performance while relying on fewer features and lower computational overhead, supporting its suitability for practical IoT botnet detection deployments.

7.1. Limitations and future research directions

Despite the strong detection performance demonstrated in this study, certain limitations should be acknowledged. The proposed framework is evaluated under static, offline conditions and does not explicitly address adversarial evasion techniques such as mimicry attacks or feature manipulation aimed at bypassing detection. In addition, concept drift caused by evolving IoT traffic patterns and emerging botnet behaviors may impact long term effectiveness. Incorporating adversarial robustness mechanisms and drift aware or adaptive learning strategies represents an important direction for future research.

Future work will examine (i) real world streaming deployment and memory/CPU profiling on representative edge hardware, (ii) adversarial robustness and resilience to concept drift (e.g., poisoning and evasive attacks), and (iii) automated feature acquisition strategies to reduce runtime feature extraction cost.

8. Conclusion

This work develops and evaluates a complete, deployment oriented pipeline for IoT botnet detection that emphasizes accuracy, interpretability, and operational efficiency. Through stepwise RFE we identify an *optimal 8-feature* subset that preserves maximal performance; through comparative hyperparameter search we demonstrate that a heuristic SmartSearch configuration produces a compact XGB model achieving identical top line accuracy (99.97%) and F_1 (99.97%) while halving inference latency relative to larger ensembles. SHAP explanations show that the classifier's decisions are driven by meaningful traffic attributes (`frame.len`, `ip.proto`, `ip.ttl`, `ip.flags`, `tcp.dstport`, `tcp.window_size`, `udp.srcport`, `udp.dstport`), reducing the likelihood that high performance arises from spurious correlations. External validation on the IoT-23/CTU subset confirms the model's generalization beyond the training distribution.

The proposed 8-feature XGB model provides a practical, high performance detector suitable for edge or gateway deployment where both accuracy and latency matter. For production use, we recommend the SmartSearch hyperparameter set (n_estimators=75, max_depth=5, learning_rate=0.25, subsample=0.9, colsample_bytree=0.9) as a starting point; deployments may further tune model size to a specific hardware budget. By combining principled feature selection, efficient hyperparameter search, and interpretable tree based learning, this work demonstrates a credible, novel route to accurate and trustworthy IoT botnet detection that is suited for real time, resource constrained operational environments. Future work will focus on real time streaming deployment, hardware aware optimization on edge devices, and improving robustness against adversarial attacks and concept drift.

CRediT authorship contribution statement

Elizabeth Antony: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Amit Kumar Roy:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

Algorithm 2 outlines the sequential steps used to clean, transform, and balance the IoT network intrusion detection dataset for machine learning analysis.

Table A.13
SmartSearch candidate hyperparameter combinations for XGB tuning.

ID	n_{est}	d_{max}	η	Sub	ColSub
S1	100	4	0.20	0.9	0.9
S2	150	3	0.30	0.8	1.0
S3	200	5	0.15	1.0	0.8
S4	50	6	0.30	0.9	0.9
S5	100	3	0.25	0.8	0.8
S6	150	4	0.20	1.0	1.0
S7	75	5	0.25	0.9	0.9
S8	125	4	0.15	0.8	1.0

Note: n_{est} = n_estimators, d_{max} = max_depth, η = learning rate, Sub = subsample, ColSub = colsample_bytree.

Algorithm 2: Preprocessing and Balancing of the Dataset

Input: Initial dataset with 2,339,621 instances and 35 attributes
Output: Refined and balanced dataset comprising 87,422 instances and 23 attributes

Array ← *RawDataset*;
Eliminate host related identifiers: `eth.src, eth.dst, ip.src, ip.dst, arp.dst.proto_ipv4, arp.src.proto_ipv4, arp.src.hw_mac, arp.dst.hw_mac, ip.addr, eth.addr`;
Remove utility fields: `frame.number, ip.tos`;
Replace NaN: numeric ← −1, categorical ← 'unknown';
Convert flags: `tcp.flags, ip.flags` → decimal;
Label-encode categorical features;
Remove duplicate records;
Array ← RandomUnderSampling(Array);
Dataset ← *Array*;
return *Dataset*;

Algorithm 3 describes the stepwise RFE procedure integrated with an XGB based classifier and F_1-score monitoring.

Algorithm 3: Stepwise RFE using XGB and F_1-score monitoring

Input: Training set (X_{train}, y_{train}), validation set (X_{val}, y_{val}), base estimator M (XGB), optional early stopping tolerance ϵ
Output: Ordered list of retained features and per iteration F_1-scores

Initialize F ← list of all features in X_{train};
Initialize arrays `FCounts` ← [ ], `F1-scores` ← [ ], and dictionary `FeatureHistory` ← {};
while $|F| \geq 1$ **do**
  Train M on $X_{train}[F], y_{train}$;
  Predict $\hat{y} \leftarrow M$.predict$(X_{val}[F])$;
  Compute $s \leftarrow F_1(y_{val}, \hat{y})$;
  Append $|F|$ to `FCounts` and s to `F1-scores`;
  Store `FeatureHistory[`$|F|$`]` ← copy of F;
  if $|F| = 1$ **then**
    break
  ;
  Compute feature importances vector $I \leftarrow M$.featureImportances;
  Identify index $j \leftarrow \arg\min I$ (least important feature);
  Remove feature F_j from F;
  if *stopping criterion satisfied (optional):*
    current s dropped more than ϵ from previous plateau **then**
    break
  ;
return `FCounts`, `F1-scores`, `FeatureHistory`;

Algorithm 4 outlines a systematic hyperparameter tuning framework for the XGB classifier using multiple search strategies to optimize model performance.

Algorithm 4: Hyperparameter Tuning for XGB Using Multiple Search Strategies

Input: Training dataset (X_{train}, y_{train}), test dataset (X_{test}, y_{test}), hyperparameter search spaces
Output: Optimized hyperparameters, trained XGB model, and test set performance metrics

Initialize hyperparameter search spaces:
  $n_estimators \leftarrow \{50, 100, 150, 200\}$
  $max_depth \leftarrow \{3, 4, 5, 6\}$
  $learning_rate \leftarrow \{0.1, 0.15, 0.2, 0.3\}$
  $subsample \leftarrow \{0.8, 0.9, 1.0\}$
  $colsample_bytree \leftarrow \{0.8, 0.9, 1.0\}$;
$Method_Results \leftarrow \emptyset$;
Method 1: Simple Randomized Search;
for $i \leftarrow 1$ **to** 15 **do**
  $params \leftarrow$ random_sample(search_spaces);
  $model \leftarrow$ train_XGB$(params, X_{train}, y_{train})$;
  $score \leftarrow$ cross_val_score$(model, X_{train}, y_{train}, cv = 3)$;
  if $score > best_score$ **then**
    $best_score \leftarrow score$;
    $best_params \leftarrow params$;
$Method_Results$[Randomized] ← $\{best_score, best_params\}$;
Method 2: Step-by-Step Grid Search;
$grid_best_params \leftarrow default_params$;
foreach *hyperparameter h in search_spaces* **do**
  foreach *value in* $search_spaces[h]$ **do**
    $test_params \leftarrow grid_best_params$ with $h = value$;
    $model \leftarrow$ train_XGB$(test_params, X_{train}, y_{train})$;
    $score \leftarrow$ cross_val_score$(model, X_{train}, y_{train}, cv = 3)$;
    if $score > grid_best_score$ **then**
      $grid_best_score \leftarrow score$;
      $grid_best_value \leftarrow value$;
  $grid_best_params[h] \leftarrow grid_best_value$;
$Method_Results$[Grid] ← $\{grid_best_score, grid_best_params\}$;
Method 3: Quick Smart Search;
foreach *params in heuristic_combinations* **do**
  $model \leftarrow$ train_XGB$(params, X_{train}, y_{train})$;
  $score \leftarrow$ cross_val_score$(model, X_{train}, y_{train}, cv = 3)$;
  if $score > smart_best_score$ **then**
    $smart_best_score \leftarrow score$;
    $smart_best_params \leftarrow params$;
$Method_Results$[Smart] ← $\{smart_best_score, smart_best_params\}$;
Final Selection and Evaluation;
Select method with highest score and lowest total time;
$final_model \leftarrow$ train_XGB$(best_params, X_{train}, y_{train})$;
$y_pred \leftarrow$ predict$(final_model, X_{test})$;
$test_metrics \leftarrow$ compute_metrics(y_{test}, y_pred);
return $best_params$, $final_model$, $test_metrics$;

A.1. SmartSearch hyperparameter search space

SmartSearch evaluates a predefined set of empirically selected XGB hyperparameter combinations (Table A.13), designed to span compact and moderate complexity ensemble configurations that are commonly effective for network traffic classification tasks. Table A.14 presents the extracted features and their brief descriptions for the IoT Network Intrusion Detection Dataset.

Data availability

Data will be made available on request.

Table A.14
List of extracted features with brief descriptions [9].

Feature name	Description
frame.number	Number of the frame within the captured pcap file
frame.len	Number of bytes within a frame
*e*th.src	Source MAC address
*e*th.dst	Destination MAC address
ip.src	Source IP address
ip.dst	Destination IP address
ip.proto	Contents of the protocol field within the IP packet
ip.len	Number of bytes in an IP packet
ip.ttl	Value of the time-to-live field in an IP packet
ip.flags	Flags field of an IP packet
ip.hdr_len	Length of the IP packet header
ip.tos	Contents of the type-of-service field in an IP packet
arp	A feature identifying if this is an ARP packet and the type of the ARP packet
arp.dst.proto_ipv4	Destination IP address as defined in the ARP packet
arp.src.proto_ipv4	Source IP address as defined in the ARP packet
tcp.flags.syn	Value of the SYN flag in the TCP header
tcp.flags.reset	Value of the RESET flag in the TCP header
icmp	A field identifying if this is an ICMP packet
tcp.checksum.status	TCP header checksum
tcp.connection.syn	TCP connection SYN field
tcp.connection.synack	TCP connection SYNACK field
tcp.dstport	Destination TCP port number
tcp.srcport	Source TCP port number
tcp.flags	Contents of all of the TCP flags field in the TCP header
tcp.len	Length of the TCP segment, as mentioned in the TCP header
tcp.time_delta	Time between this packet and the packet received before it
tcp.window_size	TCP window size
tcp.urgent_pointer	TCP urgent pointer
udp.srcport	UDP source port number
udp.dstport	UDP destination port number
tcp	Field identifying if this packet is using the TCP protocol
arp.src.hw_mac	Source MAC address as defined in the ARP packet
arp.dst.hw_mac	Destination MAC address as defined in the ARP packet
*e*th.addr	Field defining the MAC address
ip.addr	The IP address of the packet (either source or destination address)

References

[1] T. Sasi, A.H. Lashkari, R. Lu, P. Xiong, S. Iqbal, A comprehensive survey on IoT attacks: Taxonomy, detection mechanisms and challenges, J. Inf. Intell. 2 (6) (2024) 455–513, http://dx.doi.org/10.1016/j.jiixd.2023.12.001, URL https://www.sciencedirect.com/science/article/pii/S2949715923000793.

[2] O. Aouedi, T.-H. Vu, A. Sacco, D.C. Nguyen, K. Piamrat, G. Marchetto, Q.-V. Pham, A survey on intelligent Internet of Things: Applications, security, privacy, and future directions, IEEE Commun. Surv. Tutor. 27 (2) (2025) 1238–1292, http://dx.doi.org/10.1109/COMST.2024.3430368.

[3] Y.Y.F. Panduman, N. Funabiki, E.D. Fajrianti, S. Fang, S. Sukaridhoto, A survey of AI techniques in IoT applications with use case investigations in the smart environmental monitoring and analytics in real-time IoT platform, Information 15 (3) (2024) http://dx.doi.org/10.3390/info15030153, URL https://www.mdpi.com/2078-2489/15/3/153.

[4] A. Mahboubi, K. Luong, H. Aboutorab, H.T. Bui, S. Camtepe, K. Ansari, B. Barry, The evolving threat landscape of botnets: Comprehensive analysis of detection techniques in the age of artificial intelligence, Internet Things 33 (2025) 101728, http://dx.doi.org/10.1016/j.iot.2025.101728, URL https://www.sciencedirect.com/science/article/pii/S2542660525002422.

[5] H.A. Sakr, M.M. Fouda, A.F. Ashour, A. Abdelhafeez, M.I. El-Afifi, M. Refaat Abdellah, Machine learning-based detection of DDoS attacks on IoT devices in multi-energy systems, Egypt. Inform. J. 28 (2024) 100540, http://dx.doi.org/10.1016/j.eij.2024.100540, URL https://www.sciencedirect.com/science/article/pii/S1110866524001038.

[6] A. Famera, B. Hilger, S. Bhunia, P. Heil, Analyzing The Mirai IoT botnet and its recent variants: Satori, Mukashi, Moobot, and Sonic, 2025, arXiv preprint arXiv:2508.01909.

[7] Z. Zhao, Z. Li, T. Li, Z. Xu, Y. Li, Q. Xu, F. Zhang, TNT: A large-scale P2P botnet detection framework via communication topology and network traffic, IEEE Trans. Netw. 34 (2026) 562–577, http://dx.doi.org/10.1109/TON.2025.3602877.

[8] S. Pham Anh, Y. Nakamura, A baseline investigation into the evolution and prevalence of Mirai and Hajime utilizing a network telescope, IEEE Access 12 (2024) 103789–103809, http://dx.doi.org/10.1109/ACCESS.2024.3434555.

[9] M.M. Alani, BotStop: Packet-based efficient and explainable IoT botnet detection using machine learning, Comput. Commun. 193 (2022) 53–62, http://dx.doi.org/10.1016/j.comcom.2022.06.039.

[10] P. Szynkiewicz, Signature-based detection of botnet ddos attacks, in: J. Kołodziej, M. Repetto, A. Duzha (Eds.), Cybersecurity of Digital Service Chains: Challenges, Methodologies, and Tools, Springer International Publishing, Cham, 2022, pp. 120–135, http://dx.doi.org/10.1007/978-3-031-04036-8_6.

[11] F. Suthar, N. Patel, S. Khanna, B. Jakim, A signature-based botnet (Emotet) detection mechanism, Int. J. Eng. Trends Technol. 70 (2022) 185–193, http://dx.doi.org/10.14445/22315381/IJETT-V70I5P220.

[12] M. Patetta, S. Secci, S. Taktak, Line rate botnet detection with SmartNIC-embedded feature extraction, Comput. Netw. 254 (2024) 110809, http://dx.doi.org/10.1016/j.comnet.2024.110809, URL https://www.sciencedirect.com/science/article/pii/S1389128624006418.

[13] I. Apostol, M. Preda, C. Nila, I. Bica, IoT botnet anomaly detection using unsupervised deep learning, Electronics 10 (16) (2021) http://dx.doi.org/10.3390/electronics10161876, URL https://www.mdpi.com/2079-9292/10/16/1876.

[14] K. Alieyan, A. Almomani, M. Anbar, M. Alauthman, R. Abdullah, B.B. Gupta, DNS rule-based schema to botnet detection, Enterp. Inf. Syst. 15 (4) (2021) 545–564, http://dx.doi.org/10.1080/17517575.2019.1644673.

[15] S. Lysenko, K. Bobrovnikova, B. Savenko, P. Gaj, O. Savenko, Botnet detection approach based on DNS, in: IntelITSIS, 2022, pp. 400–410.

[16] H. Ostap, R. Antkiewicz, BotTROP: detection of a botnet-based threat using novel data mining algorithm, Commun. IBIMA 2022 (2022) 20, http://dx.doi.org/10.5171/2022.156851.

[17] L. Mathur, M. Raheja, P. Ahlawat, Botnet detection via mining of network traffic flow, Procedia Comput. Sci. 132 (2018) 1668–1677, http://dx.doi.org/10.1016/j.procs.2018.05.137, URL https://www.sciencedirect.com/science/article/pii/S187705091830869X. International Conference on Computational Intelligence and Data Science.

[18] A. Alharbi, K. Alsubhi, Botnet detection approach using graph-based machine learning, IEEE Access 9 (2021) 99166–99180, http://dx.doi.org/10.1109/ACCESS.2021.3094183.

[19] W. Wang, Y. Shang, Y. He, Y. Li, J. Liu, BotMark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors, Inform. Sci. 511 (2020) 284–296, http://dx.doi.org/10.1016/j.ins.2019.09.024, URL https://www.sciencedirect.com/science/article/pii/S0020025519308758.

[20] M.H. Nasir, J. Arshad, M.M. Khan, Collaborative device-level botnet detection for Internet of Things, Comput. Secur. 129 (2023) 103172, http://dx.doi.org/10.1016/j.cose.2023.103172, URL https://www.sciencedirect.com/science/article/pii/S0167404823000822.

[21] A. Lekssays, L. Landa, B. Carminati, E. Ferrari, PAutoBotCatcher: A blockchain-based privacy-preserving botnet detector for Internet of Things, Comput. Netw. 200 (2021) 108512, http://dx.doi.org/10.1016/j.comnet.2021.108512, URL https://www.sciencedirect.com/science/article/pii/S138912862100445X.

[22] T. Hasan, J. Malik, I. Bibi, W.U. Khan, F.N. Al-Wesabi, K. Dev, G. Huang, Securing industrial internet of things against botnet attacks using hybrid deep learning approach, IEEE Trans. Netw. Sci. Eng. 10 (5) (2023) 2952–2963, http://dx.doi.org/10.1109/TNSE.2022.3168533.

[23] N. Abdalgawad, A. Sajun, Y. Kaddoura, I.A. Zualkernan, F. Aloul, Generative deep learning to detect cyberattacks for the IoT-23 dataset, IEEE Access 10 (2022) 6430–6441, http://dx.doi.org/10.1109/ACCESS.2021.3140015.

[24] S. Pokhrel, R. Abbas, B. Aryal, IoT security: botnet detection in IoT using machine learning, 2021, arXiv preprint arXiv:2104.02231.

[25] M. Lefoane, I. Ghafir, S. Kabir, I.-U. Awan, Machine learning for botnet detection: An optimized feature selection approach, in: Proceedings of the 5th International Conference on Future Networks and Distributed Systems, 2021, pp. 195–200, http://dx.doi.org/10.1145/3508072.3508102.

[26] S. Srinivasan, D. P., Enhancing the security in cyber-world by detecting the botnets using ensemble classification based machine learning, Meas.: Sensors 25 (2023) 100624, http://dx.doi.org/10.1016/j.measen.2022.100624, URL https://www.sciencedirect.com/science/article/pii/S2665917422002586.

[27] J. Velasco-Mata, V. González-Castro, E. Fidalgo, E. Alegre, Real-time botnet detection on large network bandwidths using machine learning, Sci. Rep. 13 (1) (2023) 4282, http://dx.doi.org/10.1038/s41598-023-31260-0.

[28] S. Gupta, B. Singh, An intelligent multi-layer framework with SHAP integration for botnet detection and classification, Comput. Secur. 140 (2024) 103783, http://dx.doi.org/10.1016/j.cose.2024.103783, URL https://www.sciencedirect.com/science/article/pii/S0167404824000841.

[29] I. Fikriansyah Ramadhan, M. Aidiel Rachman Putra, N. Jean De La Croix, T. Ahmad, V. Eugene Kannole, K. Mtonga, E. Rweyemamu, R. Muslim Ijtihadie, SB-Net: A novel spam botnet detection scheme with two-stage cascade learner and ensemble feature selection, IEEE Open J. Commun. Soc. 6 (2025) 7237–7255, http://dx.doi.org/10.1109/OJCOMS.2025.3603425.

[30] F. Taher, M. Abdel-Salam, M. Elhoseny, I.M. El-Hasnony, Reliable machine learning model for IIoT botnet detection, IEEE Access 11 (2023) 49319–49336, http://dx.doi.org/10.1109/ACCESS.2023.3253432.

[31] M. Lefoane, I. Ghafir, S. Kabir, I.-U. Awan, Unsupervised learning for feature selection: A proposed solution for botnet detection in 5G networks, IEEE Trans. Ind. Inform. 19 (1) (2023) 921–929, http://dx.doi.org/10.1109/TII.2022.3192044.

[32] P.C. Tikekar, S.S. Sherekar, J. Kumar, An approach for detection of botnet based on machine learning classifier, SN Comput. Sci. 5 (3) (2024) 300, http://dx.doi.org/10.1007/s42979-024-02636-4.

[33] M. Ali, M.F. Mushtaq, U. Akram, D.G. Aray, M.M. Vergara, H. Karamti, I. Ashraf, Botnet detection in internet of things using stacked ensemble learning model, Sci. Rep. 15 (1) (2025) 21012, http://dx.doi.org/10.1038/s41598-025-02008-9.

[34] M. Saied, S. Guirguis, Explainable artificial intelligence for botnet detection in Internet of Things, Sci. Rep. 15 (1) (2025) 7632, http://dx.doi.org/10.1038/s41598-025-90420-6.

[35] F.E. Ayo, J.B. Awotunde, S.O. Folorunso, M.O. Adigun, S.A. Ajagbe, A genomic rule-based KNN model for fast flux botnet detection, Egypt. Inform. J. 24 (2) (2023) 313–325, http://dx.doi.org/10.1016/j.eij.2023.05.002, URL https://www.sciencedirect.com/science/article/pii/S1110866523000257.

[36] D. Gong, Y. Liu, A mechine learning approach for botnet detection using LightGBM, in: 2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA), 2022, pp. 829–833, http://dx.doi.org/10.1109/CVIDLICCEA56201.2022.9824033.

[37] W.N.H. Ibrahim, S. Anuar, A. Selamat, O. Krejcar, R. González Crespo, E. Herrera-Viedma, H. Fujita, Multilayer framework for botnet detection using machine learning algorithms, IEEE Access 9 (2021) 48753–48768, http://dx.doi.org/10.1109/ACCESS.2021.3060778.

[38] H.Y. Alshaeaa, Z.M. Ghadhban, Developing a hybrid feature selection method to detect botnet attacks in IoT devices, Kuwait J. Sci. 51 (3) (2024) 100222, http://dx.doi.org/10.1016/j.kjs.2024.100222, URL https://www.sciencedirect.com/science/article/pii/S2307410824000476.

[39] M.A. Hossain, M.S. Islam, A novel feature selection-driven ensemble learning approach for accurate botnet attack detection, Alex. Eng. J. 118 (2025) 261–277, http://dx.doi.org/10.1016/j.aej.2025.01.042, URL https://www.sciencedirect.com/science/article/pii/S1110016825000602.

[40] H. Kang, D.H. Ahn, G.M. Lee, J.D. Yoo, K.H. Park, H.K. Kim, IoT network intrusion dataset, 2019, http://dx.doi.org/10.21227/q70p-q449.

[41] S. Garcia, A. Parmisano, M.J. Erquiaga, IoT-23: A labeled dataset with malicious and benign IoT network traffic, 2020, http://dx.doi.org/10.5281/zenodo.4743746, Data set.

[42] S. García, M. Grill, J. Stiborek, A. Zunino, An empirical comparison of botnet detection methods, Comput. Secur. 45 (2014) 100–123, http://dx.doi.org/10.1016/j.cose.2014.05.011, URL https://www.sciencedirect.com/science/article/pii/S0167404814000923.

[43] N. Koroniotis, N. Moustafa, E. Sitnikova, A new network forensic framework based on deep learning for Internet of Things networks: A particle deep framework, Future Gener. Comput. Syst. 110 (2020) 91–106, http://dx.doi.org/10.1016/j.future.2020.03.042, URL https://www.sciencedirect.com/science/article/pii/S0167739X19325105.